

Become AI engineers → Week 1

LLM → AI model which can understand & generate text. (Large language model)

It can engage in a conversation

Ex:- Chatgpt → OpenAI

Claude → Anthropic

Gemini → Google

Grok → xAI

Meta AI → Meta

LLMs are build using 2 methods

① Pre training

→ thousands of GPU

\$\$\$

months of training

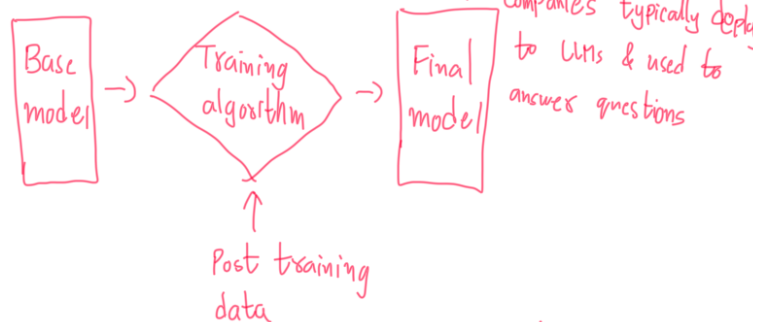


② Post training

hundreds of GPU

\$\$

Days of training



* In pretraining LLM gains knowledge to build base model

* we train base model with post training data until we get a final model.

3 steps in gathering internet data

① Crawling the internet:



few organizations like "Common crawl" which are non profitable crawls the web for free

Companies also crawl by themselves.

② Data cleaning:

HTML text content → Data cleaning → Clean data/clean text from Raw HTML text

★ We also don't want LLM to learn duplicate data.

Datasets: C4, Do1ma, RefinedWeb, FineWeb

* These 4 companies clean Rawdata & create direct datasets to train.

→ These 4 companies gives clean common crawl versions

Fine web $\rightarrow$ Data cleaning pipeline

URL
Filtering → Text
extraction → Language
Filtering → Graphs
Filtering → MinHash
dedup → C4
Filter → Custom
Filters → PII
Removal

Fine Web pipeline

③ Tokenization:

Raw text $\rightarrow$ Tokenization $\rightarrow$ long sequence of discrete numbers

Tokenization have 2 phases

① Training phase

long sequence of text

Hi I am Yashun!

text splitting

["Hi", "I", "am", "Vasun", "!"]

Building vocabulary

token	ID
Hi	1
I	2
am	3
Vorsun	4
!	5

* No. of tokens depends on length of text

- * We need to run this trained data to build vocabulary

② Inference phase

→ tokenizer stores vocabulary, when ever we ask a text to it, it converts it into sequence of numbers

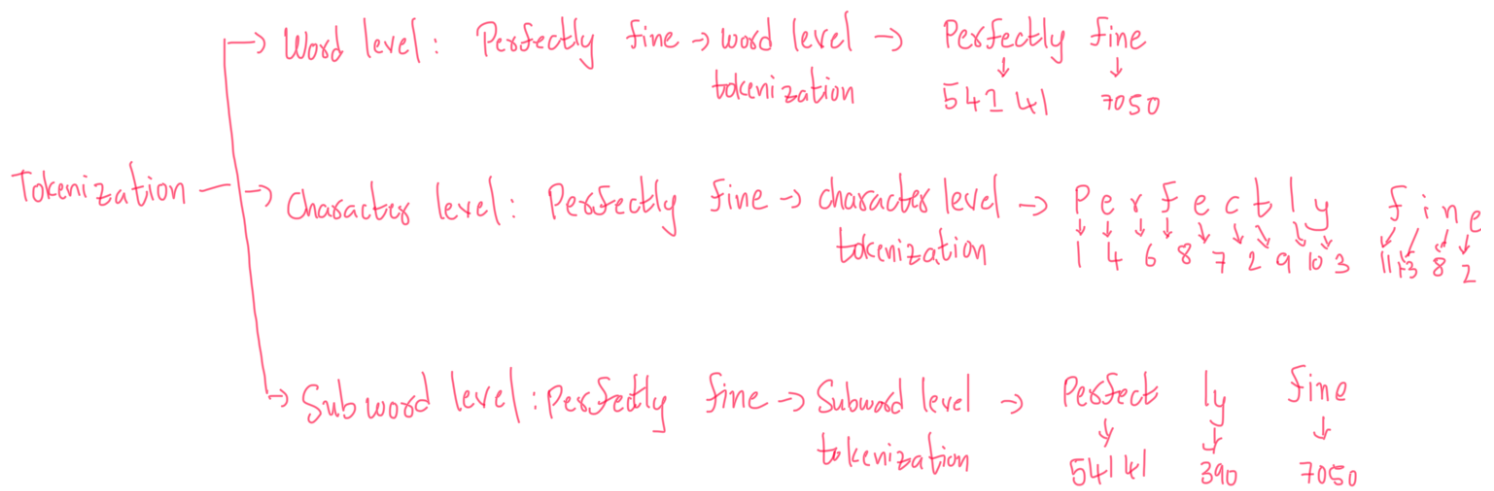
- ★ It splits a text into "sequence of tokens" & it refers to it's internal vocabulary & gives it's appropriate ID

★ It can be vice-versa, we can get text using sequence of numbers & vice versa.

tokens $\rightarrow$ ID

text splitting can be done through different algorithms & building vocabulary is a easy process.

Algorithms in text splitting & building tokens



★ Word level & character level tokenizations are no longer used in LLMs, LLMs mostly rely on "Subword level" tokenization.

★★ Byte-Pair Encoding tokenization :- Algorithm used for subword level tokenization
Most LLMs use BPE

Model Architecture:

- ① Neural networks
- ② Transformers
- ③ Transformers for text generation

① Neural Networks is a sequence of parameterized transformations that maps an input to an output.

Common layers in neural networks:

i) linear layers (linear transformation) → $y = xW + b$ } here input is x , W is internal parameter & b is bias.
 $x \rightarrow \boxed{\text{linear layers}} \rightarrow y$ } It linearly changes output Y , by using x & b parameters.

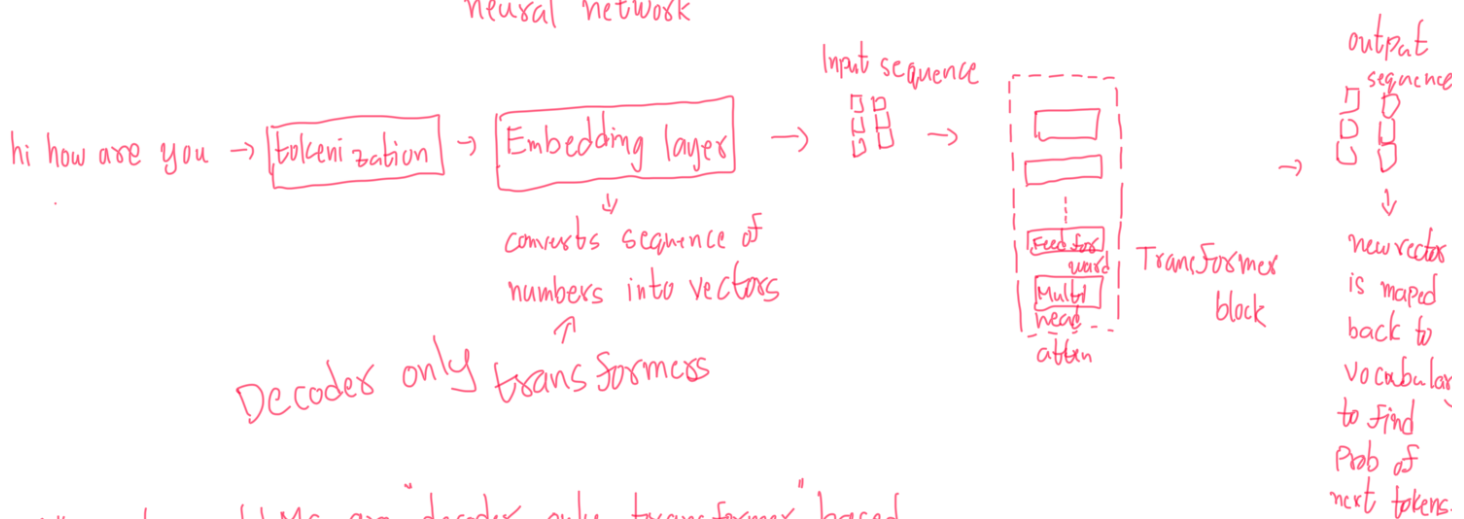
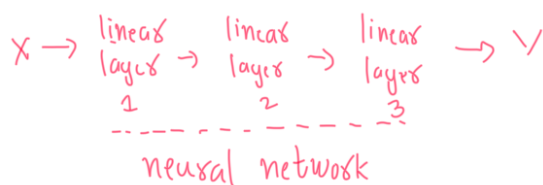
★ linear layer is a mathematical expression, that go from x to y

More layers:-

- i) Convolution layers
- iii) Normalization layers
- v) Embedding layers

- ii) Activation layers
- iv) Attention layers
- vi) Softmax layers

* Neural networks is a sequence of layers



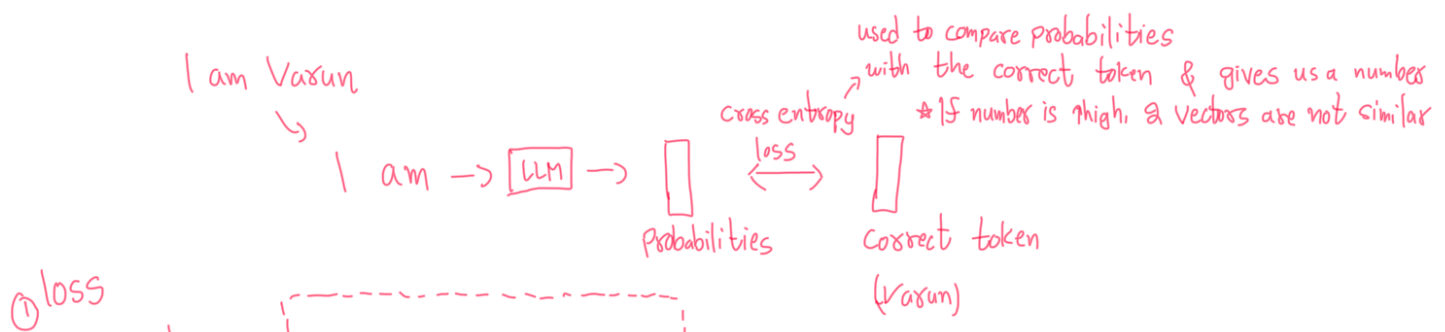
Decoder only transformers

* All modern LLMs are "decoder only transformers" based

Model training: If we do not train the model, probabilities of token are "random". [Output vectors are meaningless]

* The purpose of model training is to use processes & algorithm to tune the internal parameters

→ Each layer will keep changing parameters after we expose LLM to internet crawling



① loss

Cross entropy :- $H(P, q) = - \sum_{x \in X} P(x) \log q(x)$

② Optimization :- Updating LLMs parameters to make better predictions

*** If we test for "loss" & "optimize" data multiple times, we get highly weighted LLM ***

③ Training outcome :- A good next token predictor

I am → LLM →

"var"	0.01
"is"	0.03

*** We continuously take sample text from training data, we pass it to the model to get output probabilities & we calculate loss & we compare it to previous token & we use optimiser to update parameters. We do this multiple times with internet data so we end up with very good next token ***

④ Text generation :- Text generation is a "Iterative Process" → Mostly a for loop to generate multiple tokens

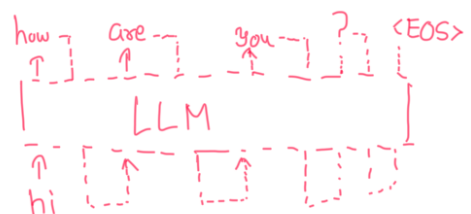
I → LLM →

--

 → am
am → LLM →

--

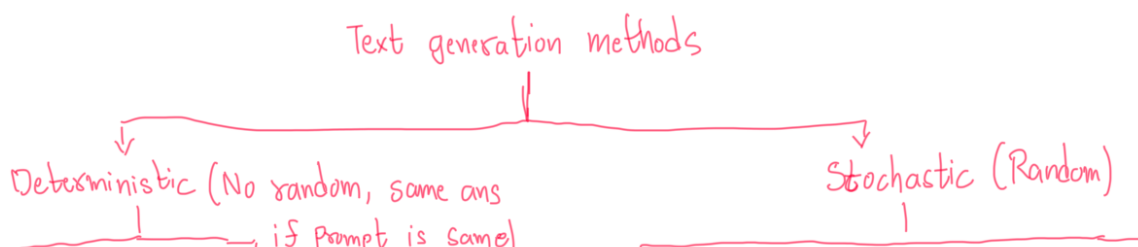
 → Varun

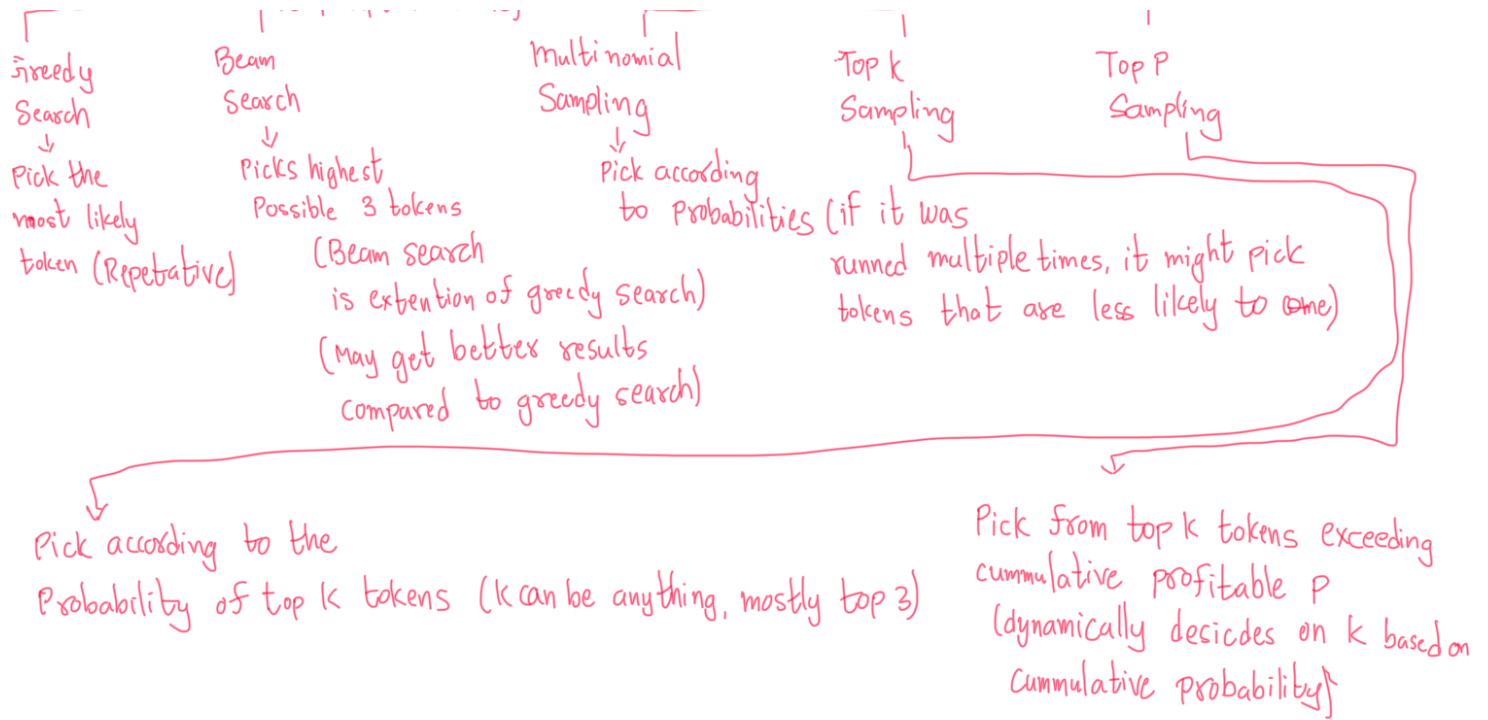


i) Generation ends if we reach end of sentence special token (ex:- <EOS>)

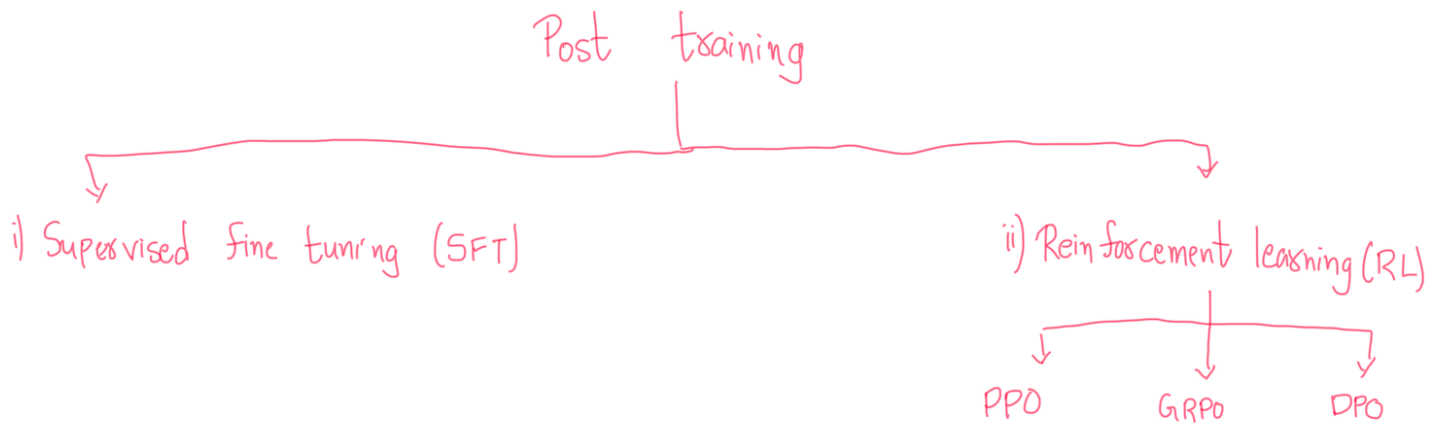
ii) If we reach maximum desired length.

Decoding Algorithms / Sampling Algorithms :- How to choose a token from Probability distribution.





Post training :



1) Supervised fine tuning/Instruction fine tuning :- Goal is to adapt a model & follow instructions

Ex:- My name is → Base Model → What to eat today? Is it good?

My name is → SFT model → My name is Vansun & you are 25 years old

Data Preparation:

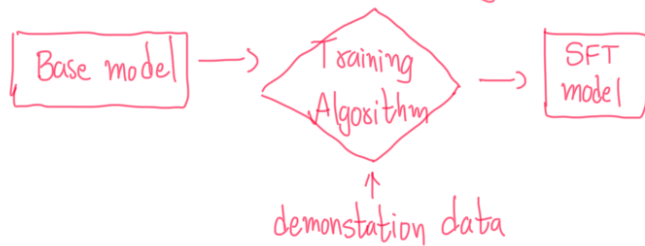
→ Having data in a Particular format < prompt, response > format

(Demonstration data)

token for prompt & token for response

Training: Similar to pretraining, just replace pretraining data with SFT data/demonstration data

Training algorithm:- Identical to pretraining



→ SFT model is not ready for deployment

② Reinforcement learning: Reinforcement learning is getting response that is preferred by humans

* SFT model after "RL", is ready for deployment *

* Reinforcement learning is letting supervised fine tuning model learn and practice *